

수행 효율성 (Performance Efficiency) 설계

목차

❖ 수행 효율성 설계 개요

❖ 활동 기반 설계

- 모델 학습 설계
- 추론 서비스 설계
- 배포 · 운영 설계

❖ 구조적 설계 패턴

기능 정확성 설계 개요

- ❖ 모델 학습·추론·운영 단계에서 연산량과 자원 사용을 최적화하여 목표 지연시간·처리량·비용 달성
- ❖ 배칭·병렬화·확장·모델 경로를 조정하여 변화하는 부하에서도 효율 유지

01

데이터 설계

- 제외단계

02

모델 학습 설계

- 모델 압축 (지식증류/가지치기/양자화)
- 학습 최적화 (병렬학습/PEFT/MoE)

03

추론서비스 설계

- Feature Store 활용
- 데이터 경량화
- 선행 추론 결과 활용
- 동적 배치 처리

03

배포 · 운영 설계

- 지연시간·자원 모니터링
- 운영 파라미터 자동 조정

구조적 설계 패턴

계층형 캐시 | 탄력적 확장 | 서빙 병렬화 | 경량-정밀 모델 라우팅 | 단계적 품질 저하

모델 학습 관점 설계

모델 학습 설계 기법 요약

유형	기법	설명
모델 압축	MT-1 지식증류	대형 모델의 정제된 지식을 소형 모델에 전수하여 경량화된 지능을 확보함으로써, 저사양 자원 환경에서도 고성능 추론이 가능하도록 유도함.
	MT-2 가지치기	중요도가 낮은 뉴런과 연결을 제거하여 모델을 희소화(Sparse)함으로써, 불필요한 가중치 이동과 연산을 줄여 자원 효율을 극대화함.
	MT-3 양자화	가중치 정밀도를 비트 수준에서 낮추어 메모리 점유를 획기적으로 줄임으로써, 전용 하드웨어 가속기에서의 연산 속도를 비약적으로 향상시킴.
학습 최적화	MT-4 병렬 학습	학습 데이터셋을 장치별로 분할하거나(Data Parallelism) 거대 모델의 연산 레이어를 여러 장치에 나누어 배치(Model Parallelism)하여 동시 처리함으로써, 단일 장치의 자원 한계를 극복하고 방대한 지능을 최단 시간에 완성하는 고속 학습 공정을 구축함
	MT-5 PEFT (Parameter-Efficient Fine-Tuning)	전체 모델을 갱신하는 대신 저차원 행렬만을 추가하여 학습함으로써, 학습에 필요한 메모리와 컴퓨팅 자원을 획기적으로 절감함.
	MT-6 MoE (Mixture of Experts)	입력 데이터에 적합한 특정 전문가 네트워크만 활성화함으로써, 지식 용량은 유지하되 실행 시의 연산 부하를 최소화함. (거대 모델 설계의 핵심)

MT-1 지식 증류

❖ 개념

- 성능이 높은 대형 **교사 모델(Teacher)**의 지식을 경량 **학생 모델(Student)**에 전달하는 모델 압축·학습 방법
- 정답 레이블뿐 아니라 교사 모델이 출력하는 클래스별 확률 분포를 학습하여, 클래스 간 유사성과 판단 경향까지 전달

❖ 기법

- 학습 데이터 → 교사 모델의 예측 결과 생성 → 학생 모델이 정답과 교사의 예측을 함께 학습
- 학생모델의 손실함수

$$L = \alpha L_{\text{정답}} + (1 - \alpha)L_{\text{교사}}$$

- ✓ $L_{\text{정답}}$: 실제 정답과 학생 모델 예측의 차이
- ✓ $L_{\text{교사}}$: 교사 모델과 학생 모델의 출력 분포 차이
- ✓ α : 실제 정답과 교사 지식의 반영 비율

❖ 예시

교사가 '사과'에 대해 Apple 90%, App 8%, Sand 0.1%로 판단했다면, 학생은 'Apple'이 정답이라는 사실뿐 아니라 'App'이 'Sand'보다 상대적으로 더 유사하다는 관계도 학습

지식 증류

❖ 주의

- 교사 모델의 오류와 편향까지 학생 모델에 전달될 수 있으므로 교사 모델의 정확성·공정성 검증이 선행되어야 함
- 교사와 학생의 구조나 성능 격차가 지나치게 크면 지식 전달이 어려울 수 있음
- 학습 중 교사와 학생을 동시에 실행하면 연산량과 메모리 사용량이 증가함.
다만 교사 출력을 미리 계산해 저장하면 동시 실행은 필수가 아님

MT-2: 가지치기

❖ 개념 및 기법

- 모델의 예측 결과에 미치는 영향이 작은 **가중치, 연결, 뉴런 또는 채널**을 제거하는 모델 압축 기법
- 필요한 정확도를 최대한 유지하면서 모델의 **파라미터 수, 저장 용량, 메모리 사용량 및 연산량**을 줄이는 것이 목적

기법	설명
가중치 크기 기반 가지치기	절댓값이 작은 가중치는 결과에 미치는 영향이 작다고 판단하여 제거
비구조적 가지치기 (Unstructured Pruning)	개별 가중치나 연결을 선택적으로 제거하여 희소한 모델 을 생성 높은 압축률을 얻을 수 있지만 희소 연산을 지원하는 실행 환경이 필요
구조적 가지치기 (Structured Pruning)	뉴런, 필터, 채널, 어텐션 헤드처럼 구조 단위로 제거 일반적인 GPU-CPU에서도 실제 속도 향상을 얻기 쉬움

원 가중치 모델

$$W = \begin{bmatrix} 0.8 & 0.02 & -0.6 \\ 0.01 & 0.5 & -0.03 \end{bmatrix}$$

↓ 절대값 < 0.05 인 가중치 제거

$$W = \begin{bmatrix} 0.8 & 0.02 & -0.6 \\ 0 & 0.5 & 0 \end{bmatrix}$$

희소한 모델

❖ 예시

학습된 모델에서 가중치 절댓값이 작은 하위 20~30%를 제거한 후, 남은 파라미터를 미세조정하여 정확도를 회복

❖ 주의

과도한 가지치기는 모델의 일반화 능력을 훼손할 수 있음

양자화(Quantization)

❖ 개념 및 기법

- 가중치와 활성화 함수 값을 표현하는 수치 정밀도를 높은 비트(예: 32비트 실수)에서 낮은 비트(예: 8비트 정수 이하)로 변환하는 정밀도 최적화 기법

기법	설명
학습 후 양자화	학습이 완료된 모델을 별도의 재학습 없이 낮은 정밀도로 변환하는 방법
동적 양자화	가중치는 미리 양자화하고 활성화 값은 추론 중에 동적으로 양자화
정적 양자화	가중치와 활성화 값을 모두 미리 양자화

❖ 예시

- FP32 모델을 배포 직전 INT8(8비트 정수) 형식으로 변환함. 이를 통해 모델 용량은 75% 감소하며, 최신 GPU의 텐서 코어(Tensor Core)를 통한 연산 속도는 수 배 향상됨.

❖ 주의

- 표현할 수 있는 숫자의 범위가 좁아지면서 '양자화 오차'로 인한 정확도 하락이 발생할 수 있음
- 모델 크기가 줄어도 배포 하드웨어와 실행 엔진이 해당 정밀도 연산을 지원하지 않으면 실제 추론 속도가 향상되지 않을 수 있음.

모델 압축

기법	핵심 개념	장점	실무 적용 지침
지식 증류 (Knowledge Distillation)	대형 모델의 지식을 소형 모델에 전수	• 근본적 파라미터 수 절감 • 소형 모델의 성능 한계 극복	"설계 초기에 결정하라." 서버급 모델을 모바일/임베디드로 이식해야 할 때 가장 먼저 검토하는 전략적 선택지임.
가지치기 (Pruning)	중요도가 낮은 뉴런/연결 제거	• 모델 파일 크기 축소 • 연산 경로의 효율화	"구조를 정밀 수술하라." 증류된 모델의 빈틈을 찾아 제거하며, 특정 가속기 환경(Sparse 가속) 시너지를 고려함.
양자화 (Quantization)	데이터 정밀도를 낮춤 (예: FP32 → INT8)	• 메모리 및 전송 부하 최소화 • 하드웨어 가속기 즉시 대응	"배포 직전 필수 공정이다." 거의 모든 플랫폼 배포 시 적용하며, 하드웨어 지원 비트(8bit, 4bit 등)를 최종 결정함.

모델 압축: 적용 가이드

❖ 적응형 레이어 관리 (Sensitivity Analysis)

- 모든 레이어를 동일한 강도로 압축하지 않음
- 정확도에 결정적인 영향을 미치는 특정 레이어(주로 첫 번째와 마지막 레이어)는 압축 대상에서 제외하거나 낮은 강도로 압축

❖ 지능의 안전마진 확보

- 목표치에 도달하더라도 정확도가 허용 한계선(SLA)에 너무 근접해 있다면, 압축 강도를 한 단계 낮추거나 더 큰 학생 모델을 선택하는 등의 '안전 마진'을 설계에 반영

❖ 수행효율성-정확도 다차원 모니터링

- 압축된 모델을 배포한 후에는 단순히 실행 속도만 보는 것이 아니라, 특정 클래스나 특정 데이터 분포에서 정확도 하락이 쏟아지지는 않는지 실시간으로 감시하여 필요 시 원본 모델로 페일오버(Fail-over)하는 로직을 구축 필요

병렬 학습

❖ 개념 및 기법

- 학습 데이터 또는 모델의 연산을 여러 GPU·장치에 분산하여 동시에 처리하는 학습 방식
 - ✓ 데이터 병렬화: 동일한 모델을 여러 GPU에 복제하고 데이터를 나누어 처리
 - ✓ 모델 병렬화: 하나의 모델을 여러 GPU에 분할하여 처리
- 실제 대규모 학습에서는 데이터 병렬화와 모델 병렬화를 함께 사용하는 혼합 병렬화도 활용

기법	설명
데이터 병렬화	• 동일한 모델 복사본을 각 GPU에 배치하고, 미니배치를 나누어 병렬 학습 • 각 GPU가 계산한 기울기를 통합한 후, 동일한 가중치로 모델을 갱신
파이프라인 병렬화	모델의 연속된 계층을 여러 GPU에 나누어 배치하고 파이프라인처럼 처리
텐서 병렬화	하나의 계층에서 수행되는 행렬 연산이나 가중치 텐서를 여러 GPU로 나누어 동시에 계산 예: LLM의 대규모 행렬 곱셈을 여러 GPU가 분할 계산한 후 결과를 결합
혼합 병렬화	데이터·파이프라인·텐서 병렬화를 조합하여 모델 크기와 장비 구성에 맞게 확장

❖ 주의

- 데이터 병렬화는 각 GPU에 전체 모델을 복제하므로, 모델 자체가 단일 GPU 메모리에 들어 가는 지 확인
- 모델 병렬화는 GPU 간 데이터 전송과 작업 의존성이 증가하므로 분할 위치와 통신량을 함께 고려해야 함

PEFT (Parameter-Efficient Fine-Tuning)

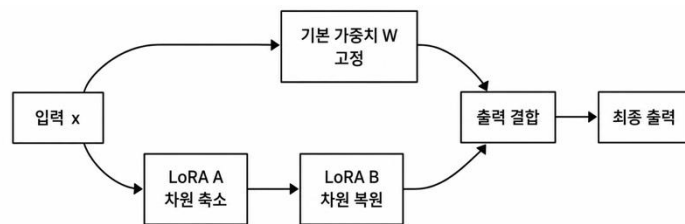
❖ 개념

- 사전학습 모델의 대부분 또는 전체 파라미터를 고정하고, 소수의 파라미터만 학습하여 특정 작업·도메인에 적응시키는 미세조정 방법
- 하나의 기본 모델에 작업별로 작은 PEFT 파라미터만 별도로 저장할 수 있어, 여러 도메인이나 사용자별 모델을 효율적으로 운영할 수 있음

❖ 대표 기법 (LoRA: Low-Rank Adaptation)

- 기존 가중치 행렬 W 는 고정하고, 가중치 변화량을 두 개의 작은 저차원 행렬 A , B 의 곱으로 표현하여 학습

$$W' = W + \Delta W, \Delta W = BA$$



- 예를 들어 $1,000 \times 1,000$ 행렬 전체를 학습하면 100만 개의 파라미터가 필요하지만, 랭크 $r=8$ 인 LoRA를 적용하면, 다음 파라미터만 학습

$$1,000 \times 8 + 8 \times 1,000 = 16,000 \text{ (전체 행렬의 약 1.6\%만 추가로 학습)}$$

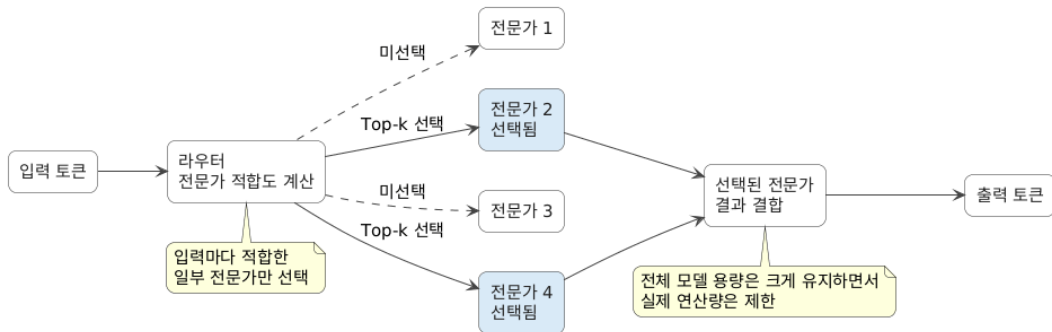
❖ 주의

- 파라미터를 지나치게 줄이면 적응 성능이 부족할 수 있으며, PEFT가 추론 모델 자체를 경량화하는 것은 아님

MoE (Mixture of Experts)

❖ 개념

- 모델 내부에 여러 개의 전문가 네트워크를 구성하고, 유입되는 토큰마다 게이트 네트워크가 가장 적합한 일부 전문가만 활성화하여 연산을 수행하는 조건부 연산 아키텍처임.



❖ 주의

- 특정 전문가에게만 작업이 쏠리는 병목 현상(Expert Bottleneck)을 방지하기 위한 부하 균형 설계가 중요
- 추론 시에는 사용하지 않는 전문가들도 메모리에 상주시켜야 하므로 막대한 VRAM 요구량을 관리해야 함.

학습 최적화

기법	핵심 개념	장점	실무 적용 지침
병렬 학습 (Parallelism)	데이터를 분산하거나 모델을 분할하여 여러 장치에서 동시 학습	• 학습 시간(TTT) 단축 • 대규모 데이터셋 수용력 확보	"인프라 활용의 기본이다." 데이터 병렬화(DP)부터 시작하고, 단일 GPU 메모리 초과 시 모델 병렬화(MP)를 검토함.
PEFT (LoRA 등)	전체 가중치 대신 일부 추가 파라미터(Adapter)만 선택적으로 업데이트	• 학습 메모리 소모 극감 • 미세 조정(Fine-tuning) 비용 절감	"적은 자원으로 고성능을 노려라." LLM 등 거대 모델을 특정 도메인에 적용시킬 때 가장 경제적이고 효과적인 선택지임.
MoE (Mixture of Experts)	입력마다 필요한 전문가 네트워크만 활성화하는 조건부 연산 구조	• 지식 용량 대비 연산량 억제 • 추론 속도와 학습 효율 동시 확보	"거대 모델의 효율을 극대화하라." 모델의 파라미터 수는 늘려 지능을 높이되, 실제 연산 자원(FLOPs)은 제한하고 싶을 때 설계함.

학습 최적화: 적용 가이드

❖ 병렬 학습의 함정

- 단순히 GPU를 많이 쓴다고 좋은 것이 아님
- 동기화 방식(Synchronous vs Asynchronous)에 따라 모델의 수렴 속도와 최종 정확도가 달라지므로, 아키텍트는 수렴 안정성을 최우선으로 인프라를 구성 필요

❖ PEFT의 한계선

- PEFT는 '적응(Adaptation)'에는 탁월하지만 '새로운 지식의 습득'에는 한계가 있음
- 도메인이 베이스 모델과 너무 이질적이라면 정확도 방어를 위해 Full Fine-tuning 방식 적용 필요

❖ MoE의 관리 복잡성

- MoE는 지능의 규모를 키우는 훌륭한 도구이지만, 라우팅 로직이 잘못되면 특정 질문에 대해 엉뚱한 전문가가 답변하는 '지능의 파편화' 발생
- 전문가 간의 성능 편차를 상시 모니터링하는 체계 필요

추론 · 운영 관점 설계

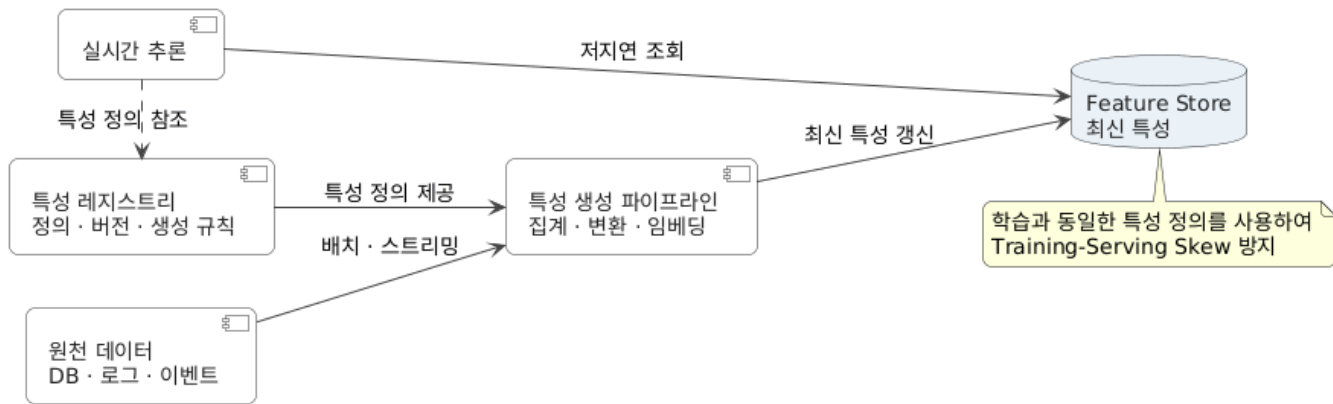
수행효율성을 위한 추론 서비스 설계 기법 요약

단계 세부 영역	핵심 설계 기술	
데이터준비	IS-1 Feature Store 활용	학습·추론에서 동일한 특징 정의를 사용하고, 사전 계산된 특징을 Feature Store에서 빠르게 조회하여 데이터 처리 지연과 중복 계산을 줄임
	IS-2 데이터 경량화	입력 데이터의 크기·차원·정밀도를 축소하여 전송량과 전처리·추론 연산량을 줄임
추론	IS-3 선행 추론 결과 활용	직전의 추론 결과를 동일·유사 요청에 재사용함으로써 응답시간과 연산 비용을 절감
	IS-4 동적 배치 처리	실시간 요청을 짧게 모아 하나의 배치로 구성하고, 목표 배치 크기 또는 최대 대기시간에 도달하면 일괄 추론하여 처리량을 향상

IS-1: Feature Store 활용

❖ 개념

- **Feature Store**는 머신러닝 모델에 사용하는 특성값(Feature)을 생성·저장·관리하고, 학습과 추론 환경에 일관되게 제공하는 데이터 관리 시스템
- 추론에 빈번하게 사용되거나 계산 복잡도가 높은 특징값 (Embedding, 통계 데이터 등)을 미리 계산하여 **Feature Store**에 관리하고, 추론 시점에는 연산 없이 즉시 조회(Look-up)



Feature Store 활용

❖ 예시

- 추천 시스템에서 사용자의 최근 7일간 구매 이력을 매번 합산하는 대신, 이미 계산된 ' 선호 카테고리 벡터'를 Feature Store에서 1ms 내에 읽어와 추론 엔진에 전달함.

❖ 주의

- 원천 데이터(Raw Data)가 변경되었을 때 Feature Store의 값이 실시간으로 동기화되지 않으면 모델의 예측 정확도가 떨어질 수 있으므로, 데이터 신선도(Freshness) 관리 정책이 병행되어야 함.
- **학습-추론 불일치:** 학습과 온라인 추론이 서로 다른 변환 코드나 데이터 원천을 사용하면 Training-Serving Skew가 발생할 수 있음
- **지연시간 의존성:** Feature Store 자체의 장애나 조회 지연이 전체 추론 서비스의 장애 또는 병목으로 이어질 수 있음

IS-2: 데이터 경량화

❖ 개념

- **데이터 경량화(Data Reduction)**는 모델 성능에 필요한 핵심 정보는 최대한 유지하면서 입력 데이터의 크기·차원·정밀도 또는 처리 범위를 줄이는 방법

기법	설명
공간 해상도 축소	이미지나 영상의 가로·세로 크기를 줄여 픽셀 수와 연산량을 감소시킵니다. 예: 4K CCTV 영상을 번호판 인식에 필요한 640×480 크기로 리사이징
관심 영역 추출	전체 데이터 중 모델이 판단에 사용하는 영역만 잘라 처리합니다. 예: CCTV 전체 화면에서 차량 또는 번호판 영역만 탐지·추출하여 인식 모델에 전달
정밀도·비트 심도 축소	픽셀, 음성 샘플 또는 센서값을 표현하는 비트 수를 줄입니다. 예: 16비트 센서 데이터를 허용 가능한 오차 범위에서 8비트로 변환
시간·공간 샘플링	영상 프레임, 음성 샘플 또는 센서 측정값의 일부만 선택합니다. 예: 30fps 영상에서 매 프레임을 처리하지 않고 초당 5프레임만 분석

❖ 주의

- 과도한 데이터 축소는 모델의 정확성을 저하시킬 수 있으므로, 성능 효율성과 정확도 사이의 임계치를 실험적으로 결정해야 함

IS-3: 선행 추론 결과 활용

❖ 개념

- 연속적으로 유입되는 입력 데이터 간의 변화가 미미한 경우, 추론을 생략하고 직전의 추론 결과를 반환하는 기법
- 시계열 데이터(영상, 센서 등)의 중복성을 활용하여 불필요한 고비용 모델 연산을 원천 차단함으로써, 시스템 자원 소모를 낮춤

❖ 예시

- 정지 상태인 자율주행 차량의 카메라 데이터가 직전 프레임과 99% 유사할 경우, 딥러닝 추론을 생략하고 "장애물 없음"이라는 직전 결과를 출력

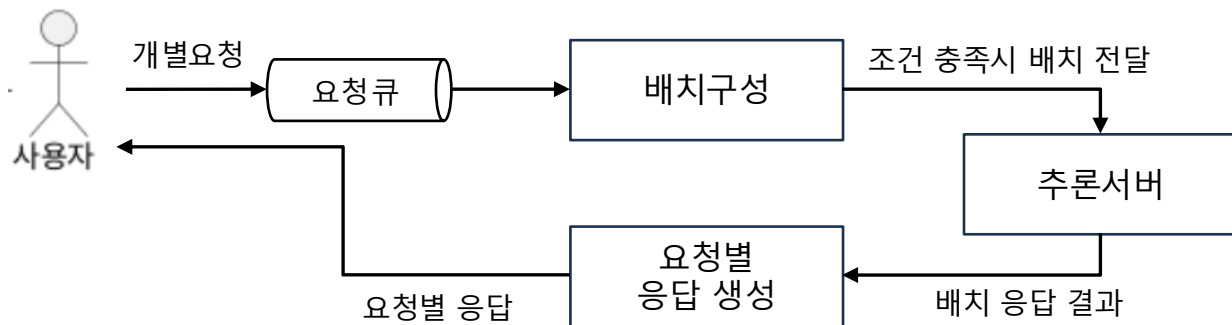
❖ 주의

- 변화를 감지하는 기준(유사도 판단 로직)이 너무 느리거나 부정확하면 오히려 전체 지연 시간이 늘어나거나 안전상 치명적인 변화를 놓칠 수 있으므로, 가벼운 알고리즘(예: Pixel Difference)을 적용해야 함.

IS-4: 동적 배칭

❖ 개념

- 실시간으로 도착하는 개별 요청을 짧은 시간 동안 모아 실행 시점마다 배치를 구성하는 온라인 서빙 기법
- 배치 크기가 목표치에 도달하면 즉시 모델에 전달



❖ 주의

- **응답 지연 증가:** 요청을 모으는 대기시간 때문에 개별 요청의 지연시간이 증가할 수 있음

배포 · 운영 관점 설계

수행효율성을 위한 배포 · 운영 설계 기법 요약

단계 세부 영역	핵심 설계 기술	
분석	DO-1 지연시간 자원 · 모니터링	종단 간·단계 별 지연시간, 처리량, 큐 길이와 CPU·GPU·메모리 사용량을 수집·분석하여 병목과 SLO 위반을 탐지
운영	DO-2 운영 파라미터 조정	모니터링 결과에 따라 배치 크기·대기시간·동시성·인스턴스 수·연산 정밀도·캐시·토큰 한도를 조정하여 성능과 비용을 목표 범위로 유지

DO-1: 지연시간 · 자원 모니터링

❖ 개념 및 기법

- **지연시간·자원 모니터링**은 운영 중인 AI 시스템의 응답 속도와 컴퓨팅 자원 사용 상태를 지속적으로 수집·분석하여 성능 저하와 자원 병목을 조기에 발견하는 운영 설계 기법
- 입력 처리, 모델 추론, 후처리 등 단계별 지연시간을 측정하고 CPU·GPU·메모리·네트워크 등의 사용량을 함께 관찰함

기법	설명
종단 간 지연시간 측정	요청이 시스템에 도착한 시점부터 최종 응답이 반환될 때까지의 전체 시간을 측정 사용자가 실제로 경험하는 응답 성능을 확인
단계별 지연시간 추적	전체 요청을 입력 전처리, 대기, 모델 추론, 후처리, 네트워크 전송 등으로 분리하여 측정 전체 지연시간 = 큐 대기시간 + 전처리시간 + 추론시간 + 후처리시간 + 통신시간
처리량과 동시 요청 감시	초당 요청 수(RPS), 초당 추론 수, 초당 토큰 수, 동시 요청 수를 측정. 부하 증가에 따른 처리 능력과 포화 시점을 파악
자원 사용률 모니터링	CPU 사용률, GPU 연산 활용률, 메모리·VRAM 사용량, 디스크 I/O와 네트워크 대역폭을 감시 특정 자원의 과부하 또는 낮은 활용률을 확인

❖ 주의

- 평균 지연시간만 보면 일부 요청의 긴 지연이 가려질 수 있으므로 p95·p99를 함께 확인해야 함
- 지나치게 상세한 로그와 추적 데이터 수집은 추가적인 연산·저장·네트워크 비용을 발생시킬 수 있음

DO-2: 수행효율성을 위한 운영 파라미터 조정

❖ 개념

- **수행효율성을 위한 운영 파라미터 조정**은 AI 서비스의 지연시간, 처리량, 자원 사용량 및 비용을 목표 수준으로 유지하도록 추론·서빙 환경의 설정값을 운영 중 조정하는 설계 기법
- 고정된 설정을 계속 사용하는 것이 아니라 트래픽, 입력 크기, 자원 상태 및 서비스 수준 목표(SLO)에 따라 파라미터를 수동 또는 자동으로 변경

❖ 예시

관찰 상태	운영 파라미터	운영 조치
큐 길이와 p95 지연 증가	인스턴스 수	추론 서버 확장
GPU 사용률이 낮고 요청이 많음	배치 크기·대기시간	배치를 확대하여 처리량 향상
GPU 메모리 부족	배치 크기·동시 처리 수	배치와 동시성 축소
저부하 상태가 지속됨	인스턴스 수	서버 축소로 비용 절감
긴 LLM 요청으로 지연 증가	입력·출력 토큰 한도	길이 제한 또는 별도 큐로 분리
SLO 위반 위험	모델·정밀도	경량 모델이나 저정밀 추론으로 전환

적용 가이드

기법	핵심 개념	장점 / 단점	효과적인 상황
ID-1. Feature Store 활용	추론에 필요한 복잡한 특징값을 미리 계산하여 중앙 저장소에 상주시키고 즉시 조회함.	(장점) 전처리 지연 시간 최소화, 데이터 일관성 확보 (단점) 데이터 신선도 관리 및 저장소 운영 비용 발생	사용자 프로필, 과거 행동 이력 등 데이터 가공 로직이 복잡하고 반복 사용되는 추천/금융 시스템
ID-2. 데이터 경량화	모델이 인식 가능한 최소 수준으로 해상도 조정, ROI 추출, 채널 압축 등을 수행함.	(장점) 데이터 전송 부하 감소, VRAM 점유율 최적화 (단점) 과도한 축소 시 모델 인식 정확도 하락	대용량 고해상도 미디어 데이터 (4K 영상 등)를 실시간으로 처리해야 하는 영상 관제 및 분석 시스템
ID-3. 선행 추론 결과 활용	입력 데이터 간 유사도를 측정하여 변화가 적을 경우 모델 추론을 건너뛰고 직전 결과를 재사용함.	(장점) 연산 자원 소모 극적 절감, 응답 연속성 보장 (단점) 변화 감지 로직의 오판 시 안전성 이슈 발생 가능	시계열 데이터의 중복성 이 높은 자율주행 차량의 정지/서행 상황이나 고정형 CCTV 객체 탐지 환경

적용 가이드

기법	핵심 개념	장점 / 단점	효과적인 상황
동적배치	실시간으로 도착하는 개별 요청을 짧게 모아 배치를 구성하고, 목표 배치 크기 또는 최대 대기 시간에 도달하면 한 번에 추론	(장점) GPU 병렬 처리 활용, 처리량 향상, 요청당 연산 비용 절감 (단점) 배치 구성 대기로 개별 응답 지연 증가, 입력 길이 편차와 트래픽 감소 시 효과 저하	요청이 지속적으로 유입되고 처리량 향상이 중요한 온라인 추론 서비스 예: 이미지 분류 API, 추천 서비스, LLM 서빙
지연시간·자원 모니터링	종단 간·단계별 지연시간과 처리량, 큐 상태, CPU·GPU·메모리·네트워크 사용량을 지속적으로 수집·분석하여 병목과 SLO 위반을 탐지	(장점) 성능 저하와 자원 병목의 조기 발견, 원인 구간 식별, 운영 조정의 근거 제공 (단점) 로그·추적에 따른 연산·저장 비용, 지표를 잘못 해석하면 불필요한 경보·조치 발생	여러 처리 단계와 자원으로 구성되어 병목 위치를 파악해야 하는 운영 시스템 예: 실시간 AI 서비스, 분산 추론 파이프라인, SLO 관리 서비스
운영 파라미터 조정	모니터링 결과와 SLO에 따라 배치 크기·대기시간·동시성·인스턴스 수·정밀도·토큰 한도 등을 수동 또는 자동으로 조정	(장점) 트래픽 변화에 대응하여 지연시간·처리량·자원 사용량·비용을 목표 범위로 유지 (단점) 잘못된 조정은 정확성 저하나 장애를 유발하며, 빈번한 변경 시 확장·축소의 제어 진동 가능	트래픽과 요청 복잡도가 크게 변하고 성능과 비용을 동적으로 최적화해야 하는 서비스 예: 자동 확장 추론 서비스, GPU 공유 환경, 입력 길이가 다양한 LLM 서비스

구조 설계 패턴

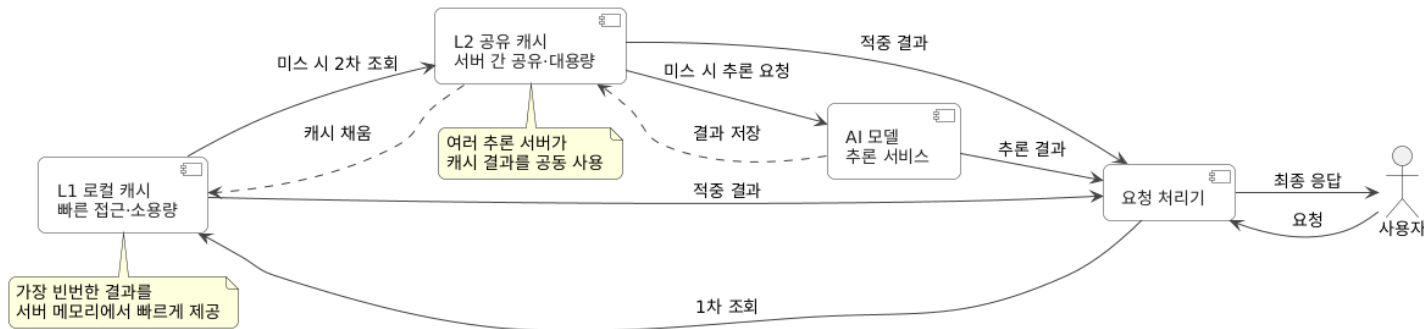
수행 효율성 설계 패턴 개요

구조적 패턴	핵심 목적
계층형 캐시 패턴	반복되는 데이터·중간 결과·추론 결과를 여러 캐시 계층에서 재사용하여 응답시간과 연산 비용 절감
탄력적 확장 패턴	부하 변화에 따라 추론 서버와 자원을 자동 증설·축소하여 성능 목표 유지와 비용 최적화
서빙 병렬화	요청이나 모델 연산을 여러 서버·GPU에 분산하여 동시에 처리함으로써 처리량 향상과 대형 모델 서빙 지원
경량-정밀 모델 라이팅 패턴	단순 요청은 경량 모델로, 어렵거나 불확실한 요청은 정밀 모델로 처리하여 정확성과 추론 비용의 균형 확보
과부하 제어 단계적 품질 저하 패턴	과부하 시 추론 품질과 연산량을 단계적으로 낮춰 서비스 중단과 연쇄 장애를 방지하고 핵심 기능 유지

계층형 캐시 패턴

❖ 개념

- 접근 속도와 저장 용량이 서로 다른 여러 캐시 계층에 중간 결과나 최종 결과를 저장하고, 가까운 계층부터 순차적으로 조회하는 패턴
- 반복되는 특성값, 임베딩, 검색 결과, 모델 출력 등을 재사용하여 연산량과 응답시간을 줄임



❖ 장점

- 반복 추론과 데이터 조회를 줄여 응답시간 단축
- GPU-CPU 연산량과 요청당 처리 비용 절감

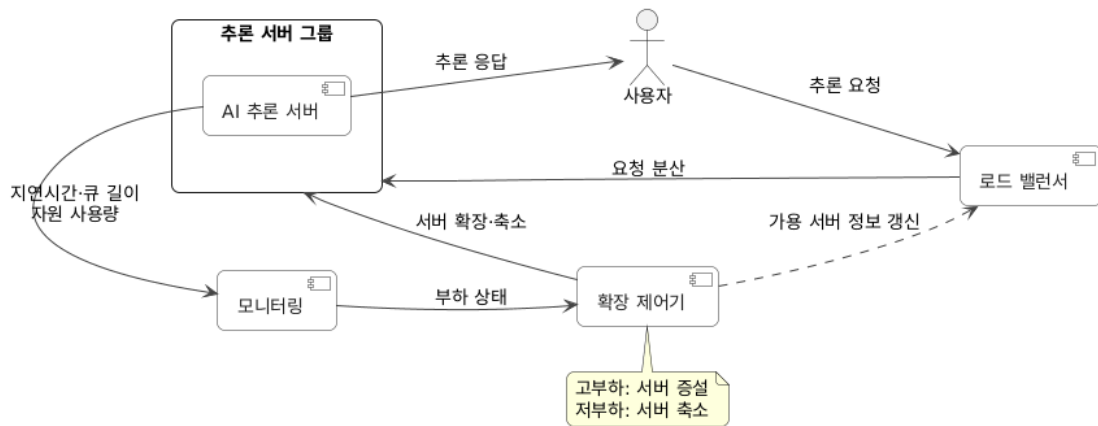
❖ 주의점

- 모델·데이터가 변경되면 오래된 결과를 반환할 수 있음
- 계층별 캐시 일관성과 무효화 정책 관리가 복잡함
- 입력이 다양하고 반복 요청이 적으면 적중률이 낮음

탄력적 확장 패턴

❖ 개념

- 트래픽과 자원 사용량에 따라 추론 서버나 컴퓨팅 자원을 자동으로 확장하거나 축소하는 패턴
- 고부하에서는 처리 용량을 늘려 SLO를 유지하고, 저부하에서는 자원을 줄여 운영 비용을 절감



❖ 장점

- 서비스 가용성과 장애 대응 능력 향상
- 과부하로 인한 지연시간 증가와 요청 실패 감소
- 모델 로딩과 GPU 초기화가 길면 확장이 늦게 적용계층별
- 최소·최대 인스턴스 수, 완충 구간, 지속시간, 축소 유예시간 등의 정책 필요

서빙 병렬화 패턴

❖ 개념

- 모델 추론 과정이나 다수의 요청을 여러 장치·인스턴스에 분산하여 동시에 처리하는 패턴
- 모델 크기와 서비스 특성에 따라 요청, 모델 내부 연산, 파이프라인 단계 등을 병렬화

❖ 병렬화 방식

- **데이터 병렬화**: 동일한 모델 복제본이 서로 다른 요청을 병렬 처리
- **텐서 병렬화**: 하나의 계층 연산을 여러 GPU에 분할
- **파이프라인 병렬화**: 모델 계층을 여러 GPU에 나누어 단계적으로 처리
- **전문가 병렬화**: MoE 모델의 전문가 모듈을 서로 다른 장치에 배치

❖ 장점

- 데이터 병렬화 시 장애가 발생한 복제본을 다른 복제본으로 대체 가능
- 단일 장치에 적재하기 어려운 대형 모델 서빙 가능
- 다수 요청을 동시에 처리하여 전체 처리량 향상

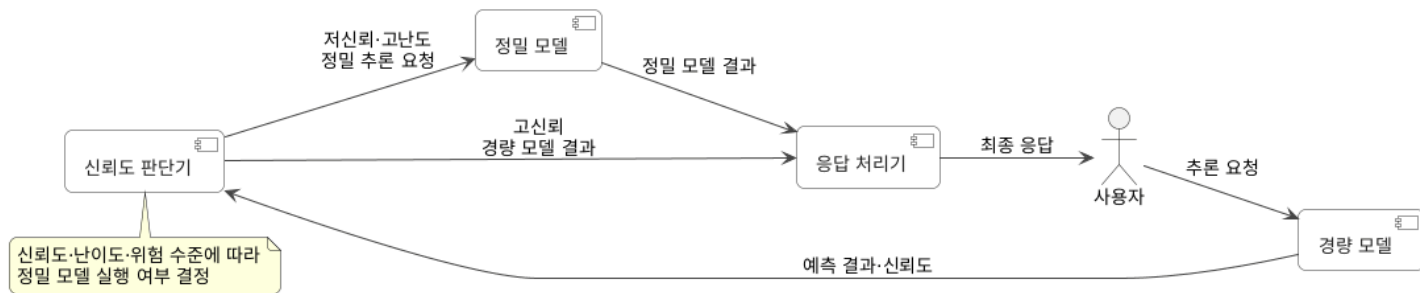
❖ 주의점

- 장치 간 통신과 결과 동기화 비용이 발생
- 작은 모델이나 낮은 트래픽에서는 통신 비용이 병렬화 효과보다 클 수 있음
- 분산 환경의 배포, 장애 복구, 버전 관리가 복잡함

경량-정밀 라우팅 패턴

❖ 개념

- 모든 요청에 고비용 정밀 모델을 적용하지 않고, 먼저 경량 모델을 실행한 뒤 난도가 높거나 신뢰도가 낮은 요청만 정밀 모델로 전달하는 패턴



❖ 장점

- 평균 추론시간과 요청당 비용 절감
- 단순 요청의 응답속도 향상
- 고비용 모델의 GPU 부하 감소
- 어려운 요청에는 정밀 모델을 적용하여 정확성 유지

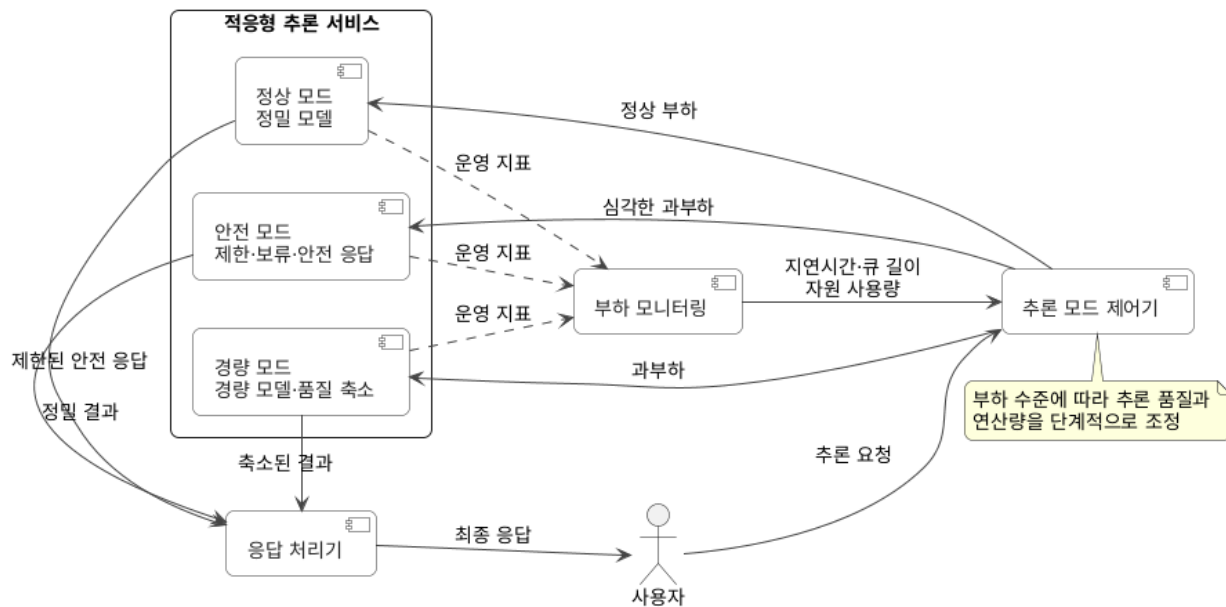
❖ 주의점

- 경량 모델이 틀렸음에도 높은 신뢰도를 보이면 정밀 모델로 전달되지 않음
- 라우팅 기준이 부정확하면 효율성과 정확성이 모두 저하될 수 있음
- 개별 모델 정확도뿐 아니라 라우팅을 포함한 종단 간 정확도·비용·지연시간 평가 필요

과부하 제어 단계적 품질 저하 패턴

❖ 개념

- 시스템 부하가 증가할 때 서비스를 완전히 중단하는 대신, 연산량이나 결과 품질을 단계적으로 낮춰 핵심 기능을 유지하는 패턴



과부하 제어 단계적 품질 저하 패턴

❖ 장점

- 과부하 상황에서도 핵심 서비스를 지속적으로 제공
- 중요 요청에 우선적으로 자원 할당 가능
- 정상 상태로 복귀할 때 품질을 단계적으로 회복 가능
- 급격한 트래픽 증가에 대한 시스템 강건성과 가용성 향상

❖ 주의점

- 경량 모델이나 축소 입력 사용으로 정확성·응답 품질이 저하될 수 있음
- 품질 저하가 허용되는 기능과 허용되지 않는 기능을 구분해야 함
- 부하 단계 전환 기준이 부적절하면 모드가 빈번하게 변경될 수 있음
- 사용자별로 다른 품질이 제공되면 일관성·공정성 문제가 발생할 수 있음

Q & A
